

종합실습 — 학사관리시스템 DB 설계 → 구축 → 조회

AI 서비스를 위한 SW 기초 Full-stack Engineering · 스마트 데이터 이해 및 활용

광주 1반 · 나용성 · 2026-07-22

1. 학사관리시스템 요구사항 (설계 방향)

대학 학사 업무의 핵심 개체인 **학과·교수·학생·강의·수강신청**을 정규화된 관계형 모델로 설계하고, PostgreSQL 위에 **DB 생성** → **스키마** → **DDL** → **DML** → **조회**까지 직접 구축한다.

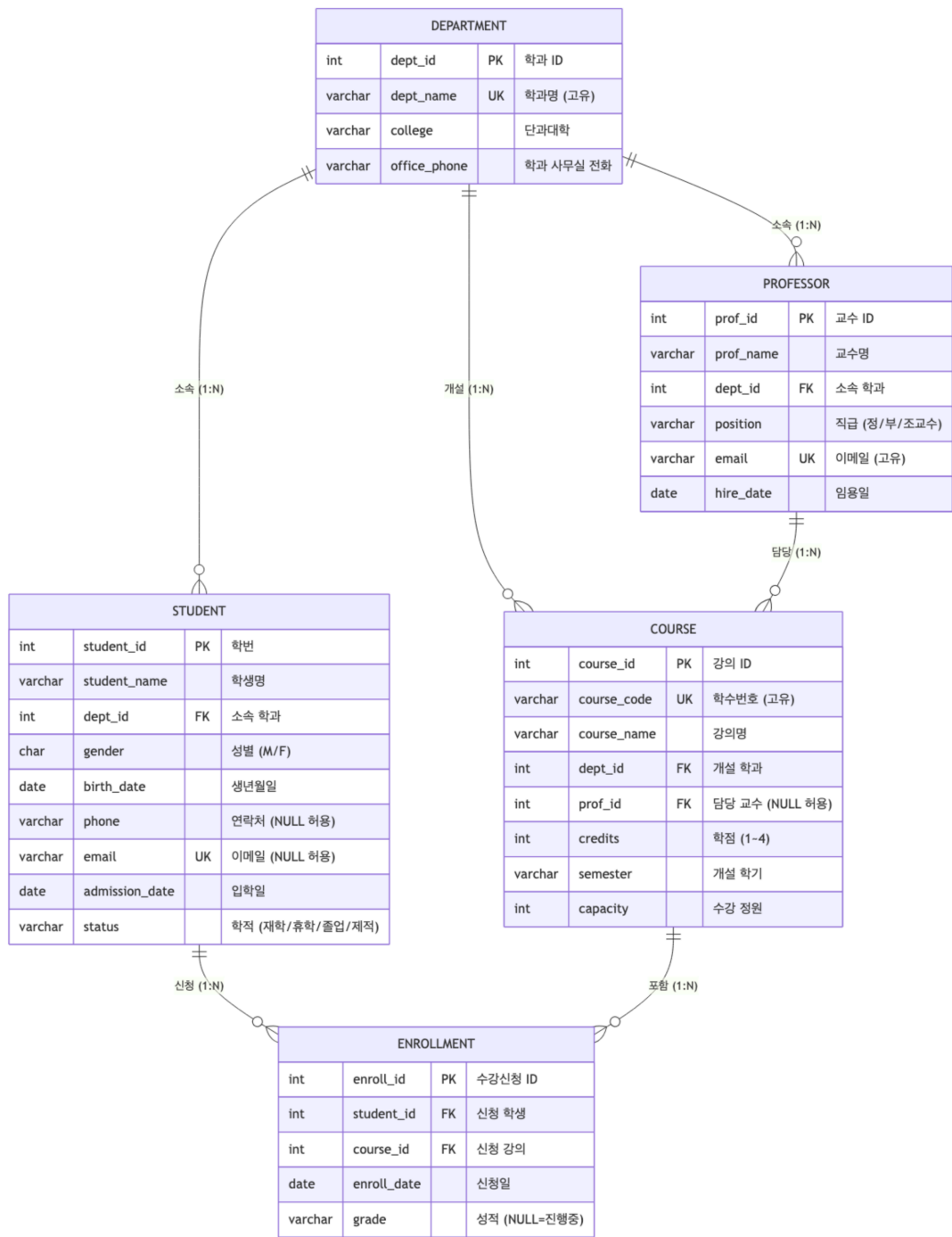
- **학과(DEPARTMENT)**는 여러 **교수·학생·강의**를 가진다. (각 1:N)
- **교수(PROFESSOR)**는 하나의 학과에 소속되며, 여러 **강의**를 담당한다. (1:N, 담당교수 미배정 강의는 NULL 허용)
- **학생(STUDENT)**은 하나의 학과에 소속되며, 여러 **강의**를 수강할 수 있다.
- **강의(COURSE)**는 하나의 개설 학과와 담당 교수를 가지며, 여러 학생이 수강한다.
- **학생 ↔ 강의**는 **N:M** 관계이므로 **수강신청(ENROLLMENT)** 교차 테이블로 해소하고, 학기별 성적을 관리한다.

무결성 설계 원칙

- **PK**: 모든 테이블에 대리키(SERIAL) 기본키 부여
- **FK**: 관계마다 외래키 지정 (department ← professor/student/course, professor/course ← enrollment 참조 체인)
- **UNIQUE**: 학과명, 교수 이메일, 학수번호, 학생 이메일 / 수강신청 (student_id + course_id) 조합 → **중복 신청 차단**
- **CHECK**: 성별(M/F), 학적(재학/휴학/졸업/제적), 직급(정·부·조교수), 학점(1~4), 성적값, 정원(>0)
- **DEFAULT / NOT NULL**: 입학일·임용일·신청일 = CURRENT_DATE, 학적 기본값 '재학' 등
- 제3정규형(3NF)을 준수해 이상현상을 방지하고, 다대다 관계는 교차 테이블로 해소

2. ERD (범례 포함)

학사관리시스템 ERD (University Academic Management System)



범례 (LEGEND) ※ 아래 박스는 다이어그램 표기 설명이며 실제 테이블이 아님	
PK / FK / UK	PK = 기본키(Primary Key), FK = 외래키(Foreign Key), UK = 고유키(UNIQUE 제약)
관계선 --o{	1:N (일대다) 관계 — 왼쪽 (하나) 쪽이 부모, 오른쪽 o{ (여러 개) 쪽이 자식
N:M 해소	STUDENT ↔ COURSE 는 다대다(N:M) 관계 → ENROLLMENT 교차 테이블로 분해하여 구현
중복 신청 방지	ENROLLMENT (student_id, course_id) 복합 UNIQUE 제약으로 동일 강의 중복 신청 차단
NULL 허용 컬럼	phone · email · prof_id(담당교수 미배정) · grade(성적 미부여=진행중)

5개 엔티티(학과·교수·학생·강의·수강신청)의 관계선이 서로 겹치지 않도록 배치. 다이어그램 하단의 범례(LEGEND) 박스에 PK/FK/UK 표기법, 관계선(||--o{ = 1:N) 의미, N:M 교차테이블 해소 방식, 중복신청 방지·NULL 허용 제약을 설명문으로 명시.

3. PostgreSQL 환경 구성 및 접속 결과 화면

설치	brew install postgresql@17 → 버전 17.10 (Homebrew)
서비스	brew services start postgresql@17
포트	5433 — 이 머신에 시스템 PostgreSQL 18이 5432를 사용 중이라 충돌 방지를 위해 postgresql.conf에서 5433으로 분리
비밀번호	ALTER USER nys WITH PASSWORD '****'; 로 계정 비밀번호 지정 (필수)
DB / Schema	university / academy

▼ 실제 접속 화면 — psql -p 5433 university 접속 후 \conninfo · \dn · \dt academy.* (docs/00_conn.png)

```
data-project/day_3/광 주 _1반 _나 용 성 _day3종 합 실 습 _v2 on ?main [?]
> psql -p 5433 university
psql (17.10 (Homebrew))
도움말을 보려면 "help"를 입력하십시오.

university=# \conninfo
접속 정보: 데이터베이스="university", 사용자="nys", 소켓="/tmp", 포트="5433".
university=# \dn
          스키마 목록
 이름      | 소유주
-----+-----
 academy   | nys
 public     | pg_database_owner
(2개 행)

university=# \dt academy.*
      릴레이션 목록
스키마 | 이름      | 형태 | 소유주
-----+-----+-----+-----
 academy | course    | 테이블 | nys
 academy | department | 테이블 | nys
 academy | enrollment | 테이블 | nys
 academy | professor  | 테이블 | nys
 academy | student    | 테이블 | nys
(5개 행)
```

4. DDL — 제약조건 포함 테이블 생성 CREATE TABLE

FK 참조 순서를 고려해 department → professor → student → course → enrollment 순으로 생성.

```
CREATE TABLE department (
  dept_id      SERIAL      PRIMARY KEY,
  dept_name    VARCHAR(50) NOT NULL UNIQUE,      -- 학과명 (고유)
  college      VARCHAR(50) NOT NULL,             -- 단과대학
```

```

        office_phone VARCHAR(20)          -- 사무실 전화 (NULL 허용)
    );

CREATE TABLE professor (
    prof_id      SERIAL          PRIMARY KEY,
    prof_name    VARCHAR(30)     NOT NULL,
    dept_id      INT             NOT NULL REFERENCES department(dept_id),
    position     VARCHAR(10)     NOT NULL DEFAULT '조교수'
                                CHECK (position IN ('정교수', '부교수', '조교수')),
    email        VARCHAR(100)    NOT NULL UNIQUE,
    hire_date    DATE            NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE student (
    student_id   SERIAL          PRIMARY KEY,          -- 학번
    student_name VARCHAR(30)     NOT NULL,
    dept_id      INT             NOT NULL REFERENCES department(dept_id),
    gender       CHAR(1)         CHECK (gender IN ('M', 'F')),
    birth_date   DATE,
    phone        VARCHAR(20),          -- NULL 허용
    email        VARCHAR(100) UNIQUE,      -- NULL 허용
    admission_date DATE            NOT NULL DEFAULT CURRENT_DATE,
    status       VARCHAR(10)     NOT NULL DEFAULT '재학'
                                CHECK (status IN ('재학', '휴학', '졸업', '제적'))
);

CREATE TABLE course (
    course_id    SERIAL          PRIMARY KEY,
    course_code  VARCHAR(10)     NOT NULL UNIQUE,          -- 학수번호
    course_name  VARCHAR(50)     NOT NULL,
    dept_id      INT             NOT NULL REFERENCES department(dept_id),
    prof_id      INT             REFERENCES professor(prof_id), -- 미배정 가능(NULL)
    credits      INT             NOT NULL DEFAULT 3 CHECK (credits BETWEEN 1 AND 4),
    semester     VARCHAR(20)     NOT NULL,
    capacity     INT             NOT NULL DEFAULT 40 CHECK (capacity > 0)
);

CREATE TABLE enrollment (
    enroll_id    SERIAL          PRIMARY KEY,
    student_id   INT             NOT NULL REFERENCES student(student_id),
    course_id    INT             NOT NULL REFERENCES course(course_id),
    enroll_date  DATE            NOT NULL DEFAULT CURRENT_DATE,
    grade        VARCHAR(2)     CHECK (grade IN ('A+', 'A', 'B+', 'B', 'C+', 'C', 'D+', 'D', 'F')),
    CONSTRAINT uq_enroll_student_course UNIQUE (student_id, course_id) -- 중복신청 방지
);

```

▼ 실제 실행 화면 — `psql -p 5433 university -f sql/03_ddl_create_table.sql` ([docs/01_ddl.png](#))


```
data-project/day_3/광 주 _1반 _나 용 성 _day3종합 실 습 _v2 on [?]main [?]
> psql -p 5433 university -f sql/04_dml_insert.sql
SET
INSERT 0 10
INSERT 0 12
INSERT 0 15
INSERT 0 12
INSERT 0 26
table_name | rows
-----+-----
course     |    12
department |    10
enrollment |    26
professor  |    12
student    |    15
(5개 행)
```

6. 조회 실습 결과 — 기초 (SELECT · WHERE · ORDER BY)

Q1. 공과대학 소속 학과 목록 (학과명 오름차순) — SELECT · WHERE · ORDER BY

▼ 실제 실행 화면 : SQL + 결과 ([docs/03.q01.png](#))

```
university=# SELECT dept_id, dept_name, college, office_phone
FROM department
WHERE college = '공 과 대 학 '
ORDER BY dept_name ASC;
 dept_id | dept_name   | college | office_phone
-----+-----+-----+-----
      3 | 기계 공 학 과 | 공 과 대 학 |
      2 | 전 자 공 학 과 | 공 과 대 학 | 062-111-1002
      1 | 컴 퓨 터 공 학 과 | 공 과 대 학 | 062-111-1001
(3개 행)
```

Q2. 현재 재학 중인 학생을 입학일 최신순으로 조회 — SELECT · WHERE · ORDER BY

▼ 실제 실행 화면 : SQL + 결과 (docs/04_q02.png)

```
university=# SELECT student_id, student_name, dept_id, admission_date, status
FROM student
WHERE status = '재 학 '
ORDER BY admission_date DESC, student_id ASC;
 student_id | student_name | dept_id | admission_date | status
-----+-----+-----+-----+-----
          1 | 김 서 연      |        1 | 2023-03-02     | 재 학
          4 | 최 윤 아      |        2 | 2023-03-02     | 재 학
          7 | 조 민 준      |        4 | 2023-03-02     | 재 학
         10 | 한 소 울      |        6 | 2023-03-02     | 재 학
         13 | 배 건 우      |        9 | 2023-03-02     | 재 학
         15 | 문 태 양      |        1 | 2023-03-02     | 재 학
          2 | 이 준 우      |        1 | 2022-03-02     | 재 학
          6 | 강 서 윤      |        3 | 2022-03-02     | 재 학
          8 | 윤 채 원      |        4 | 2021-03-02     | 재 학
         11 | 오 지 훈      |        7 | 2021-03-02     | 재 학
(10개 행)
```

Q3. 정원이 40명 초과인 강의를 정원 내림차순으로 조회 — SELECT · WHERE · ORDER BY

▼ 실제 실행 화면 : SQL + 결과 (docs/05_q03.png)

```
university=# SELECT course_code, course_name, credits, capacity, semester
FROM course
WHERE capacity > 40
ORDER BY capacity DESC;
 course_code | course_name | credits | capacity | semester
-----+-----+-----+-----+-----
    PSY101   | 심 리 학 개 론 |        3 |        70 | 2026-1
    BIZ101   | 경 영 학 원 론 |        3 |        60 | 2026-1
    EC0201   | 미 시 경 제 학 |        3 |        55 | 2026-1
    MATH101   | 선 형 대 수 학 |        3 |        50 | 2026-1
    BIZ102   | 재 무 관 리     |        3 |        50 | 2026-1
    CSE102   | 자 료 구 조     |        3 |        45 | 2026-1
    PHY101   | 일 반 물 리 학 |        3 |        45 | 2026-1
(7개 행)
```

Q4. 2015년 이후 임용된 교수를 임용일 오름차순으로 조회 — SELECT · WHERE · ORDER BY

▼ 실제 실행 화면 : SQL + 결과 (docs/06_q04.png)

```
university=# SELECT prof_id, prof_name, position, hire_date
FROM professor
WHERE hire_date >= '2015-01-01'
ORDER BY hire_date ASC;
 prof_id | prof_name | position | hire_date
-----+-----+-----+-----
      8 | 임지연   | 부교수   | 2015-03-01
     11 | 신유진   | 부교수   | 2016-03-01
      7 | 윤서준   | 조교수   | 2019-03-01
      4 | 최민석   | 조교수   | 2020-03-01
     10 | 오세훈   | 조교수   | 2021-09-01
     12 | 배준호   | 조교수   | 2022-03-01
(6개 행)
```

7. 조회 실습 결과 — 함수 (COALESCE · CASE WHEN · 날짜)

Q5. COALESCE — 연락처/이메일 미등록 항목을 대체 문자열로 표시

▼ 실제 실행 화면 : SQL + 결과 (docs/07_q05.png)

```
university=# SELECT student_name,  
                    COALESCE(phone, '(연락처 미등록)') AS phone,  
                    COALESCE(email, '(이메일 미등록)') AS email  
FROM student  
ORDER BY student_id;
```

student_name	phone	email
김 서 연	010-1001-2001	sykim@univ.ac.kr
이 준 우	010-1001-2002	jwlee@univ.ac.kr
박 하 진	(연락처 미등록)	hjpark@univ.ac.kr
최 윤 아	010-1001-2004	(이메일 미등록)
정 도 윤	010-1001-2005	dyjung@univ.ac.kr
강 서 윤	010-1001-2006	sykang@univ.ac.kr
조 민 준	(연락처 미등록)	mjcho@univ.ac.kr
윤 채 원	010-1001-2008	cwyoon@univ.ac.kr
임 지 호	010-1001-2009	jhlhim@univ.ac.kr
한 소 율	010-1001-2010	syhan@univ.ac.kr
오 지 훈	010-1001-2011	(이메일 미등록)
신 예 은	(연락처 미등록)	(이메일 미등록)
배 건 우	010-1001-2013	gwbae@univ.ac.kr
서 지 아	010-1001-2014	jaseo@univ.ac.kr
문 태 양	010-1001-2015	tymoon@univ.ac.kr

(15개 행)

Q6. CASE WHEN — 수강신청 성적을 등급 구간으로 분류 (NULL=진행중)

▼ 실제 실행 화면 : SQL + 결과 (docs/08_q06.png)

```

university=# SELECT s.student_name, c.course_name, e.grade,
CASE
    WHEN e.grade IN ('A+', 'A') THEN '우수'
    WHEN e.grade IN ('B+', 'B') THEN '양호'
    WHEN e.grade IN ('C+', 'C', 'D+', 'D') THEN '미흡'
    WHEN e.grade = 'F' THEN '낙제'
    ELSE '진행중'
END AS grade_group
FROM enrollment e
JOIN student s ON s.student_id = e.student_id
JOIN course c ON c.course_id = e.course_id
ORDER BY e.grade NULLS LAST, s.student_name;

```

student_name	course_name	grade	grade_group
김서연	선형대수학	A	우수
신예은	선형대수학	A	우수
윤채원	경영학원론	A	우수
서지아	심리학개론	A+	우수
정도윤	열역학	A+	우수
박하진	디지털논리회로	B	양호
오지훈	국문학개론	B	양호
윤채원	재무관리	B+	양호
이준우	일반물리학	B+	양호
정도윤	심리학개론	B+	양호
강서윤	일반물리학	C+	미흡
임지호	미시경제학	F	낙제
강서윤	열역학		진행중
김서연	데이터베이스		진행중
김서연	자료구조		진행중
문태양	자료구조		진행중
문태양	데이터베이스		진행중
배건우	일반물리학		진행중
오지훈	영미소설		진행중
이준우	자료구조		진행중
이준우	데이터베이스		진행중
조민준	재무관리		진행중
조민준	경영학원론		진행중
최윤아	디지털논리회로		진행중
최윤아	데이터베이스		진행중
한소울	국문학개론		진행중

(26개 행)

Q7. 날짜 함수 — 만 나이 / 재학 개월 수 계산 (AGE · DATE_PART)

▼ 실제 실행 화면 : SQL + 결과 (docs/09_q07.png)

```
university=# SELECT student_name, birth_date,
    DATE_PART('year', AGE(CURRENT_DATE, birth_date))::INT AS age,
    admission_date,
    (DATE_PART('year', AGE(CURRENT_DATE, admission_date)) * 12
    + DATE_PART('month', AGE(CURRENT_DATE, admission_date)))::INT AS months_enrolled
FROM student
WHERE birth_date IS NOT NULL
ORDER BY age DESC;
```

student_name	birth_date	age	admission_date	months_enrolled
정도윤	2001-05-18	25	2020-03-02	76
서지아	2001-09-19	24	2020-03-02	76
이준우	2003-07-22	23	2022-03-02	52
오지훈	2002-08-08	23	2021-03-02	64
임지호	2003-02-14	23	2022-03-02	52
박하진	2002-11-05	23	2021-03-02	64
윤채원	2002-12-25	23	2021-03-02	64
김서연	2004-03-15	22	2023-03-02	40
최윤아	2004-01-30	22	2023-03-02	40
강서윤	2003-09-09	22	2022-03-02	52
한소율	2004-06-06	22	2023-03-02	40
신예은	2003-10-10	22	2022-03-02	52
배건우	2004-02-02	22	2023-03-02	40
문태양	2004-11-11	21	2023-03-02	40

(14개 행)

Q8. CASE WHEN + 날짜 — 교수 근속연수 기준 시니어/미드/주니어 구분

▼ 실제 실행 화면 : SQL + 결과 (docs/10_q08.png)

```
university=# SELECT prof_name, position, hire_date,
    DATE_PART('year', AGE(CURRENT_DATE, hire_date))::INT AS years_of_service,
    CASE
        WHEN DATE_PART('year', AGE(CURRENT_DATE, hire_date)) >= 15 THEN '시니어'
        WHEN DATE_PART('year', AGE(CURRENT_DATE, hire_date)) >= 8 THEN '미드'
        ELSE '주니어'
    END AS seniority
FROM professor
ORDER BY years_of_service DESC;
```

prof_name	position	hire_date	years_of_service	seniority
강태우	정교수	2006-09-01	19	시니어
김창수	정교수	2008-03-01	18	시니어
한도경	정교수	2009-03-01	17	시니어
박도현	정교수	2010-03-01	16	시니어
정하나	부교수	2013-03-01	13	미드
이영희	부교수	2014-09-01	11	미드
임지연	부교수	2015-03-01	11	미드
신유진	부교수	2016-03-01	10	미드
윤서준	조교수	2019-03-01	7	주니어
최민석	조교수	2020-03-01	6	주니어
오세훈	조교수	2021-09-01	4	주니어
배준호	조교수	2022-03-01	4	주니어

(12개 행)

8. 조회 실습 결과 — 수강신청 교차 테이블 JOIN

Q9. 수강신청 상세 현황 — 학생·학과·강의·담당교수 5개 테이블 JOIN

▼ 실제 실행 화면 : SQL + 결과 (docs/11_q09.png)

```
university=# SELECT s.student_name, d.dept_name, c.course_code, c.course_name,
    COALESCE(p.prof_name, '(미배정)') AS professor,
    c.credits, COALESCE(e.grade, '진행중') AS grade
```

```
FROM enrollment e
```

```
JOIN student s ON s.student_id = e.student_id
```

```
JOIN department d ON d.dept_id = s.dept_id
```

```
JOIN course c ON c.course_id = e.course_id
```

```
LEFT JOIN professor p ON p.prof_id = c.prof_id
```

```
ORDER BY d.dept_name, s.student_name, c.course_code;
```

student_name	dept_name	course_code	course_name	professor	credits	grade
윤채원	경영학과	BIZ101	경영학원론	정하나	3	A
윤채원	경영학과	BIZ102	재무관리	배준호	3	B+
조민준	경영학과	BIZ101	경영학원론	정하나	3	진행중
조민준	경영학과	BIZ102	재무관리	배준호	3	진행중
임지호	경제학과	EC0201	미시경제학	강태우	3	F
한소울	국어국문학과	KOR101	국문학개론	윤서준	2	진행중
강서윤	기계공학과	ME301	열역학	최민석	3	진행중
강서윤	기계공학과	PHY101	일반물리학	오세훈	3	C+
정도윤	기계공학과	ME301	열역학	최민석	3	A+
정도윤	기계공학과	PSY101	심리학개론	신유진	3	B+
배건우	물리학과	PHY101	일반물리학	오세훈	3	진행중
신예은	수학과	MATH101	선형대수학	한도경	3	A
서지아	심리학과	PSY101	심리학개론	신유진	3	A+
오지훈	영어영문학과	ENG201	영미소설	임지연	2	진행중
오지훈	영어영문학과	KOR101	국문학개론	윤서준	2	B
박하진	전자공학과	EE201	디지털논리회로	박도현	3	B
최윤아	전자공학과	CSE101	데이터베이스	김창수	3	진행중
최윤아	전자공학과	EE201	디지털논리회로	박도현	3	진행중
김서연	컴퓨터공학과	CSE101	데이터베이스	김창수	3	진행중
김서연	컴퓨터공학과	CSE102	자료구조	이영희	3	진행중
김서연	컴퓨터공학과	MATH101	선형대수학	한도경	3	A
문태양	컴퓨터공학과	CSE101	데이터베이스	김창수	3	진행중
문태양	컴퓨터공학과	CSE102	자료구조	이영희	3	진행중
이준우	컴퓨터공학과	CSE101	데이터베이스	김창수	3	진행중
이준우	컴퓨터공학과	CSE102	자료구조	이영희	3	진행중
이준우	컴퓨터공학과	PHY101	일반물리학	오세훈	3	B+

(26개 행)

Q10. 강의별 수강 인원 집계 — LEFT JOIN + GROUP BY (정원 대비 신청률)

▼ 실제 실행 화면 : SQL + 결과 (docs/12_q10.png)

```
university=# SELECT c.course_code, c.course_name, c.capacity,
COUNT(e.enroll_id) AS enrolled,
ROUND(COUNT(e.enroll_id) * 100.0 / c.capacity, 1) AS fill_rate_pct
FROM course c
LEFT JOIN enrollment e ON e.course_id = c.course_id
GROUP BY c.course_id, c.course_code, c.course_name, c.capacity
ORDER BY enrolled DESC, c.course_code;
```

course_code	course_name	capacity	enrolled	fill_rate_pct
CSE101	데이터 베이스	40	4	10.0
CSE102	자료구조	45	3	6.7
PHY101	일반물리학	45	3	6.7
BIZ101	경영학원론	60	2	3.3
BIZ102	재무관리	50	2	4.0
EE201	디지털논리회로	35	2	5.7
KOR101	국문학개론	40	2	5.0
MATH101	선형대수학	50	2	4.0
ME301	열역학	30	2	6.7
PSY101	심리학개론	70	2	2.9
EC0201	미시경제학	55	1	1.8
ENG201	영미소설	40	1	2.5

(12개 행)

Q11. 학생별 신청 과목 수 / 취득 학점 / 진행중 과목 수 (조건부 집계)

▼ 실제 실행 화면 : SQL + 결과 (docs/13_q11.png)

```
university=# SELECT s.student_name,
COUNT(e.enroll_id) AS total_courses,
SUM(CASE WHEN e.grade IS NOT NULL THEN c.credits ELSE 0 END) AS earned_credits,
SUM(CASE WHEN e.grade IS NULL THEN 1 ELSE 0 END) AS in_progress
FROM student s
JOIN enrollment e ON e.student_id = s.student_id
JOIN course c ON c.course_id = e.course_id
GROUP BY s.student_id, s.student_name
ORDER BY earned_credits DESC, total_courses DESC;
 student_name | total_courses | earned_credits | in_progress
-----+-----+-----+-----
정 도 윤      |              2 |              6 |           0
윤 체 원      |              2 |              6 |           0
김 서 연      |              3 |              3 |           2
이 준 우      |              3 |              3 |           2
강 서 윤      |              2 |              3 |           1
임 지 호      |              1 |              3 |           0
서 지 아      |              1 |              3 |           0
박 하 진      |              1 |              3 |           0
신 예 은      |              1 |              3 |           0
오 지 훈      |              2 |              2 |           1
문 태 양      |              2 |              0 |           2
조 민 준      |              2 |              0 |           2
최 윤 아      |              2 |              0 |           2
배 건 우      |              1 |              0 |           1
한 소 율      |              1 |              0 |           1
(15개 행 )
```

Q12. 학과별 개설 강의 수 및 소속 학생 수 (스칼라 서브쿼리 다중 집계)

▼ 실제 실행 화면 : SQL + 결과 (docs/14_q12.png)

```
university=# SELECT d.dept_name, d.college,
(SELECT COUNT(*) FROM course c WHERE c.dept_id = d.dept_id) AS course_count,
(SELECT COUNT(*) FROM student s WHERE s.dept_id = d.dept_id) AS student_count
FROM department d
ORDER BY student_count DESC, d.dept_name;
 dept_name | college | course_count | student_count
-----+-----+-----+-----
컴 퓨 터 공 학 과 | 공 과 대 학 |              2 |              3
경 영 학 과 | 경 영 대 학 |              2 |              2
기 계 공 학 과 | 공 과 대 학 |              1 |              2
전 자 공 학 과 | 공 과 대 학 |              1 |              2
경 제 학 과 | 경 영 대 학 |              1 |              1
국 어 국 문 학 과 | 인 문 대 학 |              1 |              1
물 리 학 과 | 자 연 과 학 대 학 |              1 |              1
수 학 과 | 자 연 과 학 대 학 |              1 |              1
심 리 학 과 | 사 회 과 학 대 학 |              1 |              1
영 어 영 문 학 과 | 인 문 대 학 |              1 |              1
(10개 행 )
```

— 이상으로 학사관리시스템 DB 설계(ERD) → 구축(DDL/DML) → 조회(WHERE·함수·CASE WHEN·JOIN) 종합실습을 완료함. 모든 결과 화면은 직접 실행 후 캡처하였으며(docs/), 재현용 SQL은 sql/, 실행 로그 텍스트는 results/ 폴더에 포함.